

Regression Project

Connor Spencer, Lucas Koscho, Vishnu Jariwala

April 28th, 2025

I pledge my honor that I have abided by the Stevens Honor System.

```
# Set seed
CWID = 20030846
personal = CWID %% 10000
set.seed(personal)

# Load required packages
library(leaps)

# Load the data
insurance = read.csv('insurance.csv')

# Convert categories to factors
insurance$Region = as.factor(insurance$Region)
insurance$CarType = as.factor(insurance$CarType)
insurance$FuelType = as.factor(insurance$FuelType)
insurance$MaritalStatus = as.factor(insurance$MaritalStatus)

# Visualize data
summary(insurance)
```

##	X	AnnualPremium	Region	CarType	FuelType
##	Min. : 1.00	Min. : 464.0	Rural :106	Other:96	Diesel : 58
##	1st Qu.: 88.25	1st Qu.: 963.1	Suburban:146	Sedan:90	Electric: 40
##	Median :175.50	Median :1132.8	Urban : 98	SUV :75	Gas :173
##	Mean :175.50	Mean :1162.7		Truck:89	Hybrid : 79
##	3rd Qu.:262.75	3rd Qu.:1317.2			
##	Max. :350.00	Max. :3027.4			
##	MaritalStatus	DriverAge	CarAge	MilesDriven	Accidents
##	Divorced: 44	Min. :18.00	Min. : 0.00	Min. : 5105	Min. :0.00
##	Married :170	1st Qu.:33.00	1st Qu.: 5.00	1st Qu.:11823	1st Qu.:0.00
##	Single :102	Median :46.00	Median :10.00	Median :13673	Median :0.00
##	Widowed : 34	Mean :46.66	Mean :10.02	Mean :13874	Mean :0.42
##		3rd Qu.:61.00	3rd Qu.:15.00	3rd Qu.:15996	3rd Qu.:1.00
##		Max. :75.00	Max. :20.00	Max. :23919	Max. :4.00
##	CreditScore	AnnualIncome	CarMaintenanceCost	DriverResponsibilityScore	
##	Min. :371.0	Min. : 0	Min. :11.39	Min. : -40.583	
##	1st Qu.:547.0	1st Qu.: 35316	1st Qu.:26.04	1st Qu.: 8.428	
##	Median :602.0	Median : 48960	Median :30.15	Median : 35.888	
##	Mean :600.4	Mean : 49538	Mean :30.73	Mean : 38.124	

```
## 3rd Qu.:652.0 3rd Qu.: 63723 3rd Qu.:35.72 3rd Qu.: 66.659
## Max. :800.0 Max. :100217 Max. :53.63 Max. :115.468
## DiscretionarySpending PolicyLoyaltyMonths LocalCarTrend
## Min. : 7.00 Min. : 1.043 Min. :0.6471
## 1st Qu.:17.00 1st Qu.: 3.965 1st Qu.:2.0833
## Median :20.00 Median : 8.571 Median :2.7072
## Mean :20.09 Mean : 12.907 Mean :2.8326
## 3rd Qu.:23.00 3rd Qu.: 17.242 3rd Qu.:3.6163
## Max. :33.00 Max. :105.708 Max. :5.3600
```

```
# Function for k-fold splitting
kfold = function(insuranceTest) {
  # Shuffle data
  insuranceTest = insuranceTest[sample(nrow(insuranceTest)), ]

  # Create 10 equally sized folds
  folds = cut(seq(1, nrow(insuranceTest)), breaks = 10, labels = FALSE)

  # Initialize list to store data for each fold
  fold_data = list()

  # Perform cross validation
  for (i in 1:10) {
    # Segment data by fold
    testIndexes = which(folds == i, arr.ind = TRUE)
    test_data = insuranceTest[testIndexes, ]
    train_data = insuranceTest[-testIndexes, ]

    # Store train and test data for each fold
    fold_data[[i]] = list(train_data = train_data, test_data = test_data)
  }
  return(fold_data)
}
```

```
# Use k-fold splitting on insurance data
split_data = kfold(insurance)

# View training and test data for first fold
summary(split_data[[1]]$train_data)
```

```
##      X      AnnualPremium      Region      CarType      FuelType
## Min. : 1.0 Min. : 464 Rural : 94 Other:90 Diesel : 51
## 1st Qu.: 90.5 1st Qu.: 959 Suburban:129 Sedan:80 Electric: 37
## Median :180.0 Median :1132 Urban : 92 SUV :64 Gas :153
## Mean :178.1 Mean :1161 Truck:81 Hybrid : 74
## 3rd Qu.:266.5 3rd Qu.:1315
## Max. :350.0 Max. :3027
## MaritalStatus DriverAge      CarAge      MilesDriven      Accidents
## Divorced: 40 Min. :18.00 Min. : 0.00 Min. : 5105 Min. :0.000
## Married :152 1st Qu.:33.00 1st Qu.: 5.00 1st Qu.:11828 1st Qu.:0.000
## Single : 92 Median :46.00 Median :10.00 Median :13620 Median :0.000
## Widowed : 31 Mean :46.94 Mean : 9.86 Mean :13803 Mean :0.419
## 3rd Qu.:61.50 3rd Qu.:15.00 3rd Qu.:15812 3rd Qu.:1.000
```

```
##           Max.      :75.00    Max.      :20.00    Max.      :23919    Max.      :4.000
## CreditScore AnnualIncome CarMaintenanceCost DriverResponsibilityScore
## Min.      :395.0    Min.      : 0    Min.      :11.39    Min.      : -40.583
## 1st Qu.   :546.5    1st Qu.   :35994    1st Qu.   :26.05    1st Qu.   : 9.134
## Median    :603.0    Median    :49433    Median    :30.00    Median    : 37.259
## Mean      :600.6    Mean      :49986    Mean      :30.51    Mean      : 38.792
## 3rd Qu.   :651.5    3rd Qu.   :63842    3rd Qu.   :35.44    3rd Qu.   :67.769
## Max.      :800.0    Max.      :100217    Max.      :53.63    Max.      :115.468
## DiscretionarySpending PolicyLoyaltyMonths LocalCarTrend
## Min.      : 7.00    Min.      : 1.043    Min.      :0.6471
## 1st Qu.   :17.00    1st Qu.   : 3.965    1st Qu.   :2.0752
## Median    :20.00    Median    : 8.762    Median    :2.6675
## Mean      :20.24    Mean      :13.220    Mean      :2.8342
## 3rd Qu.   :24.00    3rd Qu.   :17.300    3rd Qu.   :3.6311
## Max.      :33.00    Max.      :105.708    Max.      :5.3600
```

```
summary(split_data[[1]]$test_data)
```

```
##           X           AnnualPremium           Region           CarType           FuelType
## Min.      : 3.0    Min.      : 808.9    Rural      :12    Other: 6    Diesel  : 7
## 1st Qu.   :69.0    1st Qu.   :995.1    Suburban:17    Sedan:10    Electric: 3
## Median    :149.0    Median   :1136.6    Urban      : 6    SUV :11    Gas      :20
## Mean      :152.2    Mean      :1179.6                                Truck: 8    Hybrid  : 5
## 3rd Qu.   :221.0    3rd Qu.   :1339.5
## Max.      :345.0    Max.      :1750.4
## MaritalStatus DriverAge           CarAge           MilesDriven
## Divorced: 4    Min.      :19.00    Min.      : 0.00    Min.      : 6467
## Married :18    1st Qu.   :31.00    1st Qu.   : 6.00    1st Qu.   :12014
## Single  :10    Median    :46.00    Median    :12.00    Median    :14129
## Widowed : 3    Mean      :44.09    Mean      :11.49    Mean      :14517
##           3rd Qu.   :54.00    3rd Qu.   :18.00    3rd Qu.   :17402
##           Max.      :73.00    Max.      :20.00    Max.      :22534
## Accidents CreditScore AnnualIncome CarMaintenanceCost
## Min.      :0.0000    Min.      :371.0    Min.      :10783    Min.      :15.34
## 1st Qu.   :0.0000    1st Qu.   :551.0    1st Qu.   :29493    1st Qu.   :26.13
## Median    :0.0000    Median    :596.0    Median    :46285    Median    :33.52
## Mean      :0.4286    Mean      :599.5    Mean      :45500    Mean      :32.63
## 3rd Qu.   :1.0000    3rd Qu.   :654.5    3rd Qu.   :59028    3rd Qu.   :39.08
## Max.      :2.0000    Max.      :786.0    Max.      :83456    Max.      :49.87
## DriverResponsibilityScore DiscretionarySpending PolicyLoyaltyMonths
## Min.      : -30.921    Min.      : 9.00    Min.      : 1.576
## 1st Qu.   : 6.999    1st Qu.   :17.00    1st Qu.   : 3.984
## Median    :27.254    Median    :19.00    Median    : 6.864
## Mean      :32.109    Mean      :18.71    Mean      :10.092
## 3rd Qu.   :57.744    3rd Qu.   :21.00    3rd Qu.   :13.585
## Max.      :91.968    Max.      :26.00    Max.      :27.624
## LocalCarTrend
## Min.      :1.176
## 1st Qu.   :2.262
## Median    :2.819
## Mean      :2.819
## 3rd Qu.   :3.413
## Max.      :4.371
```

```

# Store all full models
full_models = list()

# Initialize R-squared for full model
R_squared_full = numeric(length(split_data))

# Initialize AIC for full model
AIC_full = numeric(length(split_data))

# Initialize RMSE for full model
rmse_full = numeric(length(split_data))

# Access training and test data for each fold
for (i in 1:length(split_data)) {
  train_data = split_data[[i]]$train_data
  test_data = split_data[[i]]$test_data

  # Fit full model using all predictors
  full_model = lm(AnnualPremium ~ ., data = train_data)
  full_models[[i]] = full_model
  R_squared_full[i] = summary(full_model)$adj.r.squared
  AIC_full[i] = AIC(full_model)

  # Test full model against test data
  pred_test_full = predict(full_model, newdata = test_data)
  rmse_full[i] = sqrt(mean((pred_test_full-test_data$AnnualPremium)^2))
}
summary(full_models[[1]])

```

```

##
## Call:
## lm(formula = AnnualPremium ~ ., data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -403.82  -85.83  -10.64   89.34  708.95
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.211e+03  1.613e+02   7.507 7.43e-13 ***
## X              -1.098e-01  7.921e-02  -1.386   0.167
## RegionSuburban    4.386e+01  2.125e+01   2.063   0.040 *
## RegionUrban       1.711e+02  2.667e+01   6.418 5.59e-10 ***
## CarTypeSedan     -7.344e+00  2.214e+01  -0.332   0.740
## CarTypeSUV        1.322e+02  2.438e+01   5.424 1.23e-07 ***
## CarTypeTruck      1.816e+02  3.401e+01   5.340 1.88e-07 ***
## FuelTypeElectric  4.281e+00  3.024e+01   0.142   0.888
## FuelTypeGas       1.427e+01  2.228e+01   0.640   0.522
## FuelTypeHybrid    3.820e+01  2.518e+01   1.517   0.130
## MaritalStatusMarried -2.996e+01  2.514e+01  -1.192   0.234
## MaritalStatusSingle -1.029e+01  2.842e+01  -0.362   0.718
## MaritalStatusWidowed -2.898e+01  3.351e+01  -0.865   0.388
## DriverAge        -5.988e-01  3.243e+00  -0.185   0.854

```

```
## CarAge                -5.271e-01  2.705e+00  -0.195    0.846
## MilesDriven            1.118e-02  1.563e-02   0.715    0.475
## Accidents              3.321e+02  1.087e+01  30.560 < 2e-16 ***
## CreditScore            -9.304e-01  1.592e-01  -5.843  1.37e-08 ***
## AnnualIncome           1.428e-03  1.137e-03   1.255    0.210
## CarMaintenanceCost     3.421e+00  7.714e+00   0.443    0.658
## DriverResponsibilityScore -1.289e+00  1.596e+00  -0.808    0.420
## DiscretionarySpending  -2.611e+00  2.783e+00  -0.938    0.349
## PolicyLoyaltyMonths     2.769e+00  5.731e-01   4.831  2.20e-06 ***
## LocalCarTrend           8.095e+00  1.555e+01   0.521    0.603
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 133.7 on 291 degrees of freedom
## Multiple R-squared:  0.8397, Adjusted R-squared:  0.827
## F-statistic: 66.27 on 23 and 291 DF,  p-value: < 2.2e-16
```

```
# Store all stepwise models
step_models = list()

# Initialize R-squared for stepwise model
R_squared_step = numeric(length(split_data))

# Initialize AIC for stepwise model
AIC_step = numeric(length(split_data))

# Initialize RMSE for stepwise model
rmse_step = numeric(length(split_data))

# Access training and test data for each fold
for (i in 1:length(split_data)) {
  train_data = split_data[[i]]$train_data
  test_data = split_data[[i]]$test_data

  # Fit full model using all predictors
  full_model = lm(AnnualPremium ~ ., data = train_data)

  # Stepwise selection
  step_model = step(full_model, direction = 'both', trace = FALSE)
  step_models[[i]] = step_model
  R_squared_step[i] = summary(step_model)$adj.r.squared
  AIC_step[i] = AIC(step_model)

  # Test stepwise model against test data
  pred_test_step = predict(step_model, newdata = test_data)
  rmse_step[i] = sqrt(mean((pred_test_step-test_data$AnnualPremium)^2))
}
summary(step_models[[1]])
```

```
##
## Call:
## lm(formula = AnnualPremium ~ Region + CarType + MilesDriven +
##     Accidents + CreditScore + AnnualIncome + DriverResponsibilityScore +
##     PolicyLoyaltyMonths, data = train_data)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -419.09 -102.01   -7.74   88.60  692.59
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.139e+03  8.299e+01  13.730 < 2e-16 ***
## RegionSuburban    5.180e+01  1.815e+01   2.854  0.00461 **
## RegionUrban      1.785e+02  1.990e+01   8.971 < 2e-16 ***
## CarTypeSedan     -1.462e+01  2.065e+01  -0.708  0.47927
## CarTypeSUV       1.376e+02  2.199e+01   6.255  1.35e-09 ***
## CarTypeTruck     1.944e+02  2.054e+01   9.465 < 2e-16 ***
## MilesDriven      1.828e-02  2.528e-03   7.231  3.93e-12 ***
## Accidents        3.326e+02  1.065e+01  31.241 < 2e-16 ***
## CreditScore      -9.028e-01  1.559e-01  -5.792  1.74e-08 ***
## AnnualIncome     1.014e-03  5.945e-04   1.706  0.08910 .
## DriverResponsibilityScore -1.587e+00  2.206e-01  -7.194  4.97e-12 ***
## PolicyLoyaltyMonths  2.663e+00  5.589e-01   4.765  2.93e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 133.1 on 303 degrees of freedom
## Multiple R-squared:  0.8346, Adjusted R-squared:  0.8286
## F-statistic: 139 on 11 and 303 DF, p-value: < 2.2e-16
```

```
# Store all selected models
models_selected = list()

# Initialize R-squared for selected model
R_squared_select = numeric(length(split_data))

# Initialize AIC for selected model
AIC_select = numeric(length(split_data))

# Initialize RMSE for selected model
rmse_select = numeric(length(split_data))

# Access training and test data for each fold
for (i in 1:length(split_data)) {
  train_data = split_data[[i]]$train_data
  test_data = split_data[[i]]$test_data

  # Regsubsets to find best subset of predictors
  subset_model = regsubsets(AnnualPremium ~ ., data = train_data, nvmax = ncol(train_data)-1)
  reg_summary = summary(subset_model)
  num_predictors = which.max(reg_summary$adjr2)

  # Find which predictors were selected
  selected_predict = names(which(reg_summary$which[num_predictors, -1]))
  selected_predict = selected_predict[selected_predict %in% names(train_data)]

  # Refit model using selected predictors
  formula_selected = as.formula(paste('AnnualPremium ~', paste(selected_predict, collapse = ' + ')))
```

```

model_selected = lm(formula_selected, data = train_data)
models_selected[[i]] = model_selected
R_squared_select[i] = summary(model_selected)$adj.r.squared
AIC_select[i] = AIC(model_selected)

# Test selected model against test data
pred_test_select = predict(model_selected, newdata = test_data)
rmse_select[i] = sqrt(mean((pred_test_select-test_data$AnnualPremium)^2))
}
summary(models_selected[[1]])

```

```

##
## Call:
## lm(formula = formula_selected, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -415.4  -113.0    -7.4   117.6   737.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1351.18376   101.29600   13.339 < 2e-16 ***
## X              -0.20009     0.09925   -2.016  0.04467 *
## DriverAge      -3.34299     0.59187   -5.648 3.69e-08 ***
## MilesDriven     0.02452     0.00330    7.429 1.09e-12 ***
## Accidents      342.40105    14.03243   24.401 < 2e-16 ***
## CreditScore    -0.84613     0.13456   -6.288 1.10e-09 ***
## PolicyLoyaltyMonths 2.15567     0.73435    2.935 0.00358 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 176.1 on 308 degrees of freedom
## Multiple R-squared:  0.7056, Adjusted R-squared:  0.6999
## F-statistic: 123.1 on 6 and 308 DF, p-value: < 2.2e-16

```

```

# Compare R-squared across models
mean(R_squared_full)

```

```
## [1] 0.8274501
```

```
mean(R_squared_step)
```

```
## [1] 0.8291094
```

```
mean(R_squared_select)
```

```
## [1] 0.7664746
```

```
print('')
```

```
## [1] ""
```

```
# Compare AIC across models
mean(AIC_full)
```

```
## [1] 3987.4
```

```
mean(AIC_step)
```

```
## [1] 3974.433
```

```
mean(AIC_select)
```

```
## [1] 4063.738
```

```
print('')
```

```
## [1] ""
```

```
# Compare RMSE across models
mean(rmse_full)
```

```
## [1] 135.3614
```

```
mean(rmse_step)
```

```
## [1] 136.2705
```

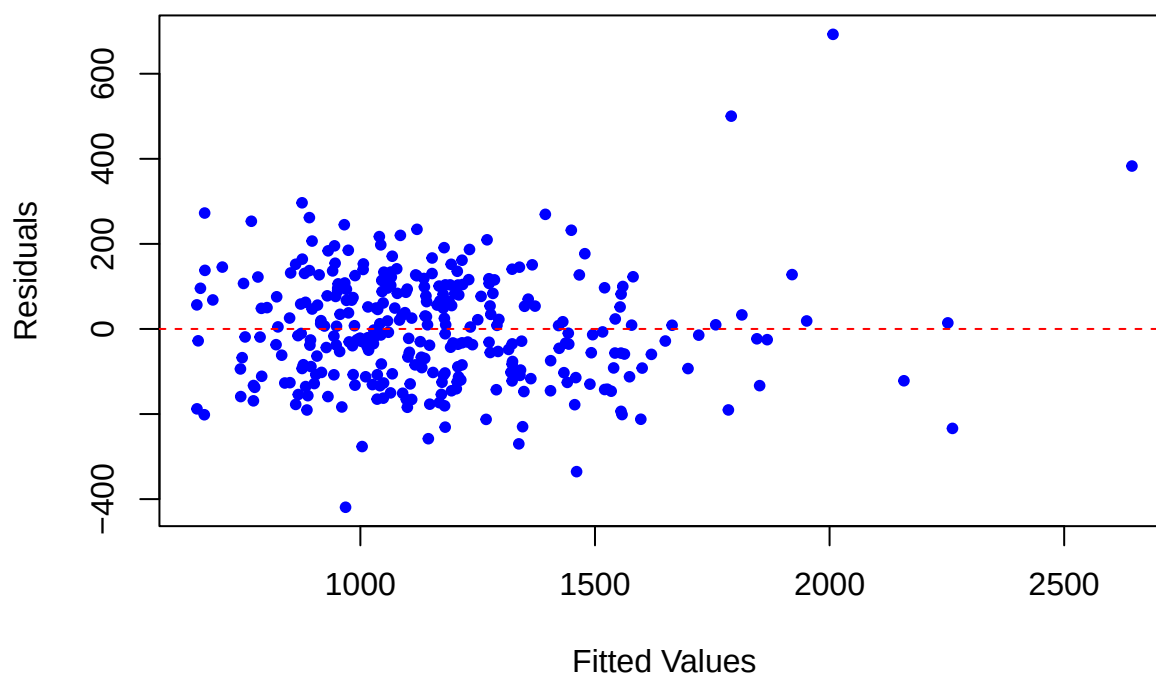
```
mean(rmse_select)
```

```
## [1] 157.0385
```

The Stepwise Selection Model has the highest R-squared value (0.8291), the lowest AIC (3974.43), and the second lowest root Mean Squared Error (136.27). Therefore, we will be using this as our model.

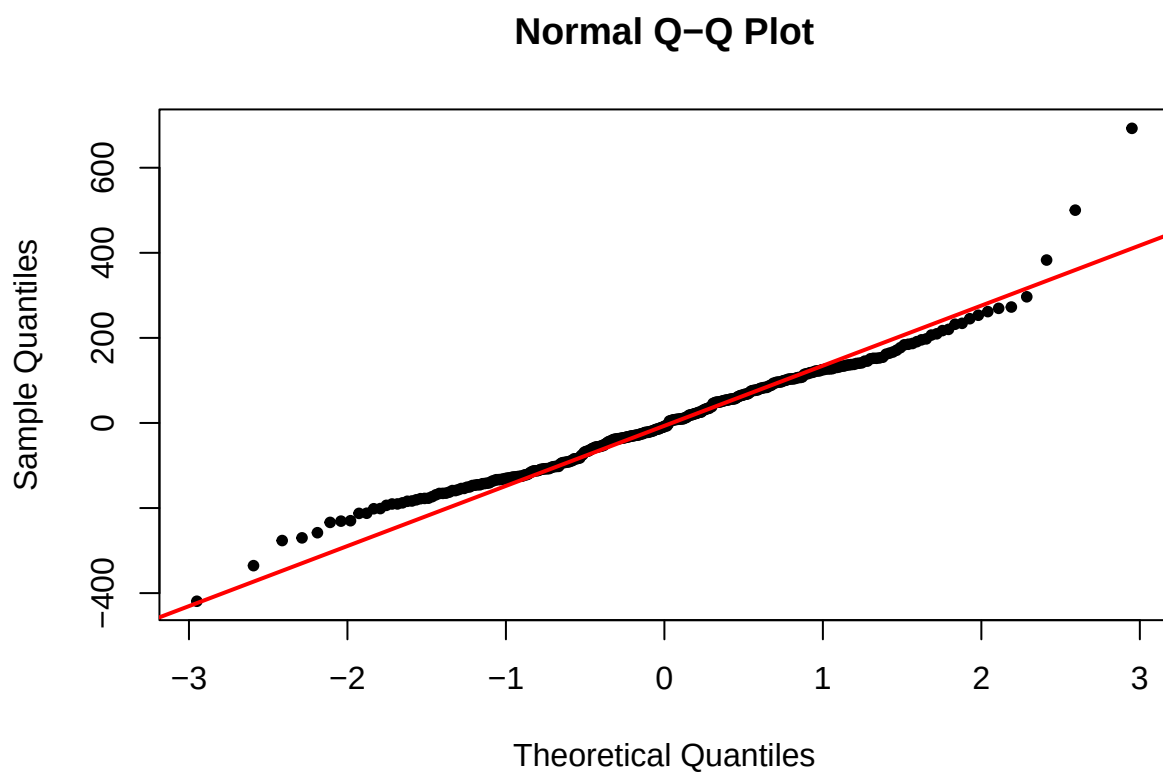
```
# Residual Plot
plot(
  step_models[[1]]$fitted.values,
  step_models[[1]]$residuals,
  xlab = "Fitted Values",
  ylab = "Residuals",
  main = "Residuals vs Fitted Values",
  pch = 20,
  col = "blue"
)
abline(h = 0, lty = 2, col = "red")
```

Residuals vs Fitted Values



Since our residuals on the graph create no pattern, it tells us that our linearity and variance of our model is accurate.

```
# QQ Plot  
qqnorm(step_models[[1]]$residuals, pch = 20, main = "Normal Q-Q Plot")  
qqline(step_models[[1]]$residuals, col = "red", lwd = 2)
```



The plot follows a mostly straight line, with only slight deviations at the tails, indicating that there is normality among our residuals.